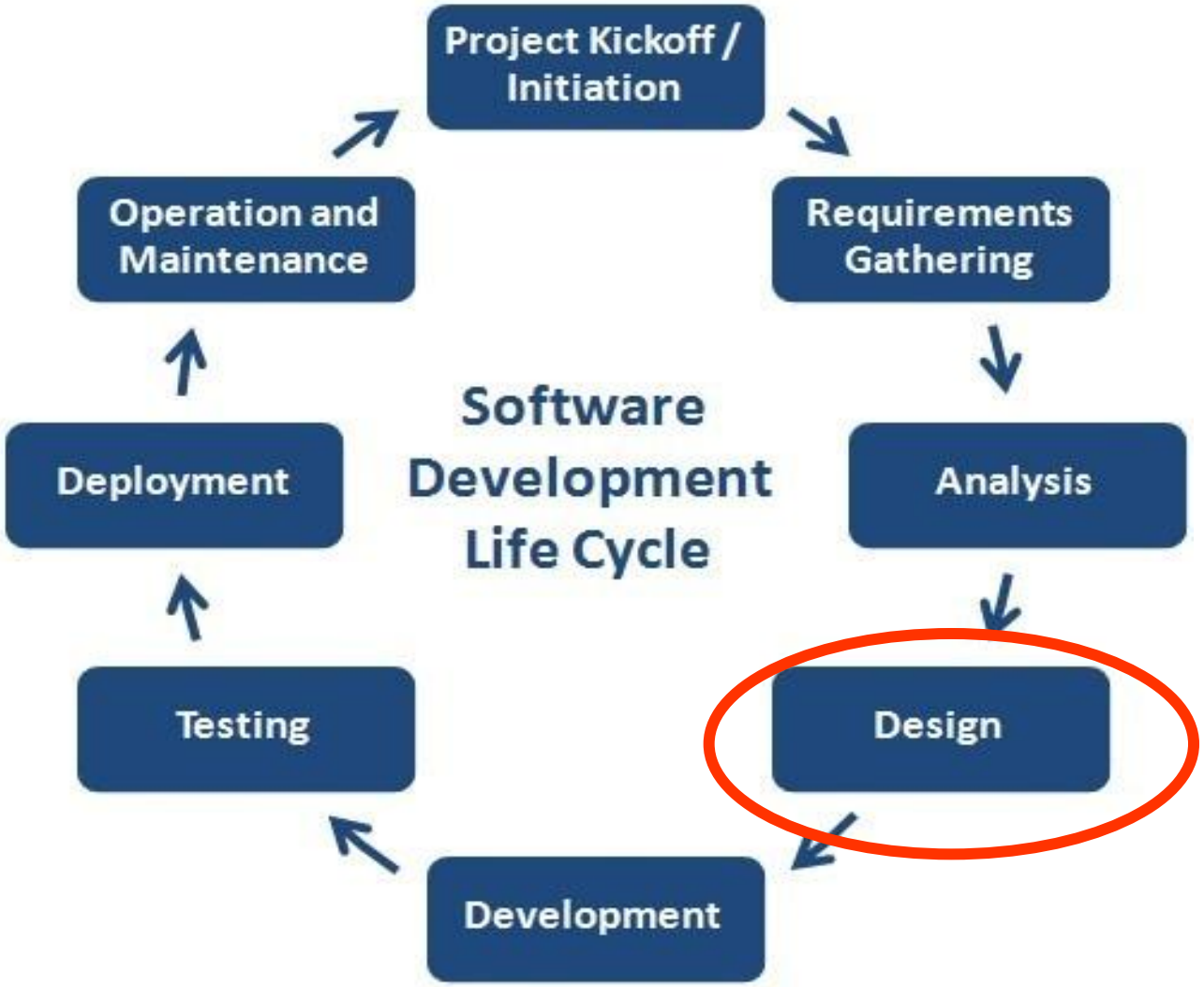
3	**Unit 3 Design Concepts** Design Process, Design Concepts, Design model, Architectural Design-Software Architecture, Component level design- Introduction, UI design-Golden rules, pattern-based design-Design pattern, web application Design, mobile application.	10

Software Development Life Cycle (SDLC) model:

SDLC Phases:



https://www.learntek.org/blog/sdlc-phases/

Imp

Software design: "A PLAN OR DRAWING".

- Software Design -- An iterative process transforming requirements into a "blueprint" for constructing the software.
- Design allows you to model the system or product that is to be built.
- Software design is the process of implementing software solutions to one or more sets of problems

Good software design should exhibit:

Firmness: A program should not have any bugs that inhibit its function.

Commodity: A program should be suitable for the purposes for which it was intended.

Delight: The experience of using the program should be pleasurable one.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Why Design is important ?

The importance of software design can be stated with a single word — quality.

- Design is the place where quality is fostered in SE.
- It provides the representation of software that can be assessed for quality.
- Design is the only way that can accurately translate a customer 's requirements into a finished software product.
- It serves as the foundation for all the software engineering and software support activities that follow.

- Without design, we risk building an unstable system :
- one that will fail when small changes are made;
- one that may be difficult to test Quality;
- one whose quality cannot be assessed until late in the software process, when time is short, and many dollars have already been spent.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts :

Design is the place where **creativity** rules—where stakeholder requirements, business needs, and technical considerations all come together in the formulation of a product or system.

Design creates a representation or model of the software.
But unlike the requirements model (that focuses on describing required data, function, and behavior), the design model provides **detail about software data structures, architecture, interfaces, and components that are necessary to implement the system.**

Design is the place where software quality is established.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design within the Context of Software Engineering:

- Software design is technical kernel of software engineering and is applied regardless of the software process model that is used.
- Beginning, once software requirements have been analyzed and modeled, software design is the last software engineering action within the modeling activity and sets the stage for construction (code generation and testing).
- Each of the elements of the requirements model provides information that is necessary to create the four design models required for a complete specification of design.
- The flow of information during software design is illustrated in following figure.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design within the Context of Software Engineering:

- ❑ The requirements model, manifested by
- ❑ **scenario-based,**
- ❑ **class-based,**
- ❑ **flow-oriented, and**
- ❑ **behavioral elements**, feed the design task.

✓ Software design model consists of Four designs:

✓ Data/Class Design

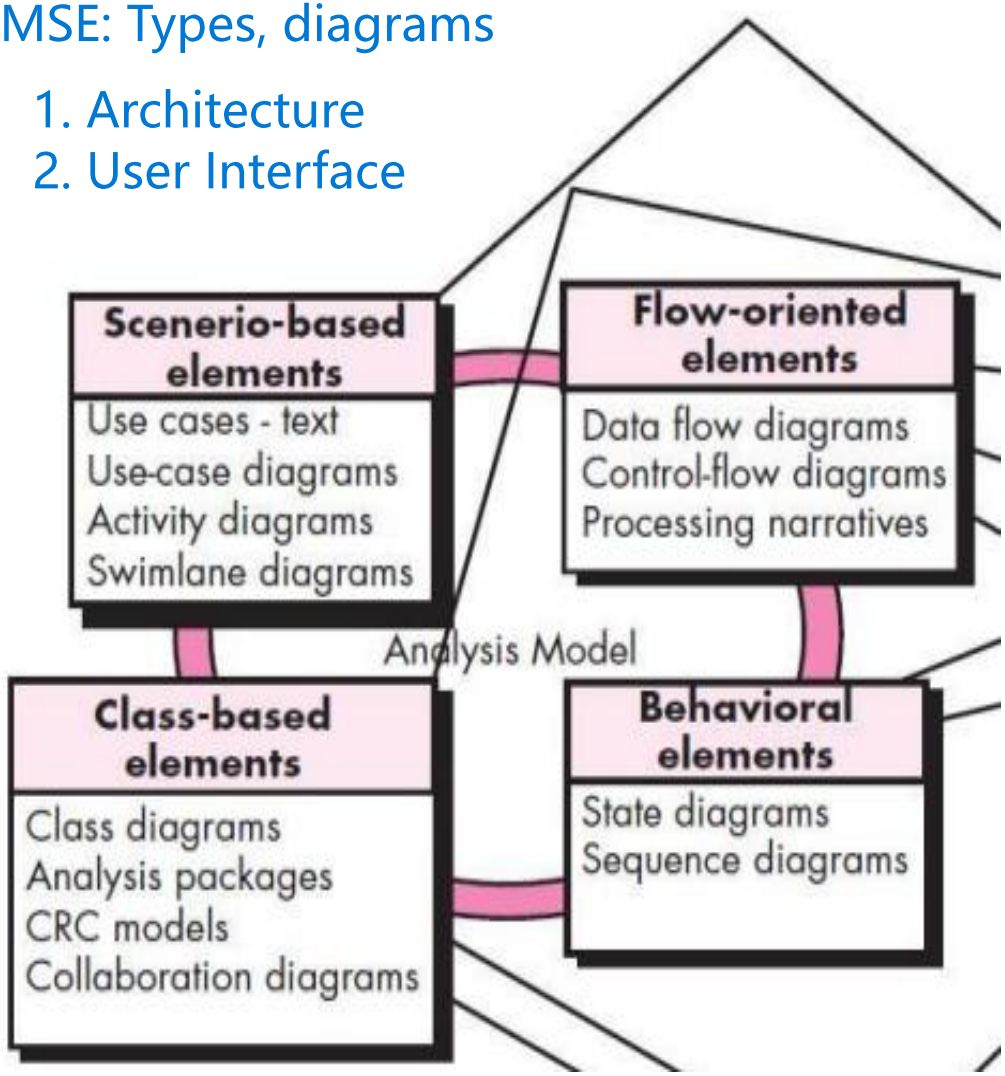
✓ Architecture

✓ Interface

✓ Component level

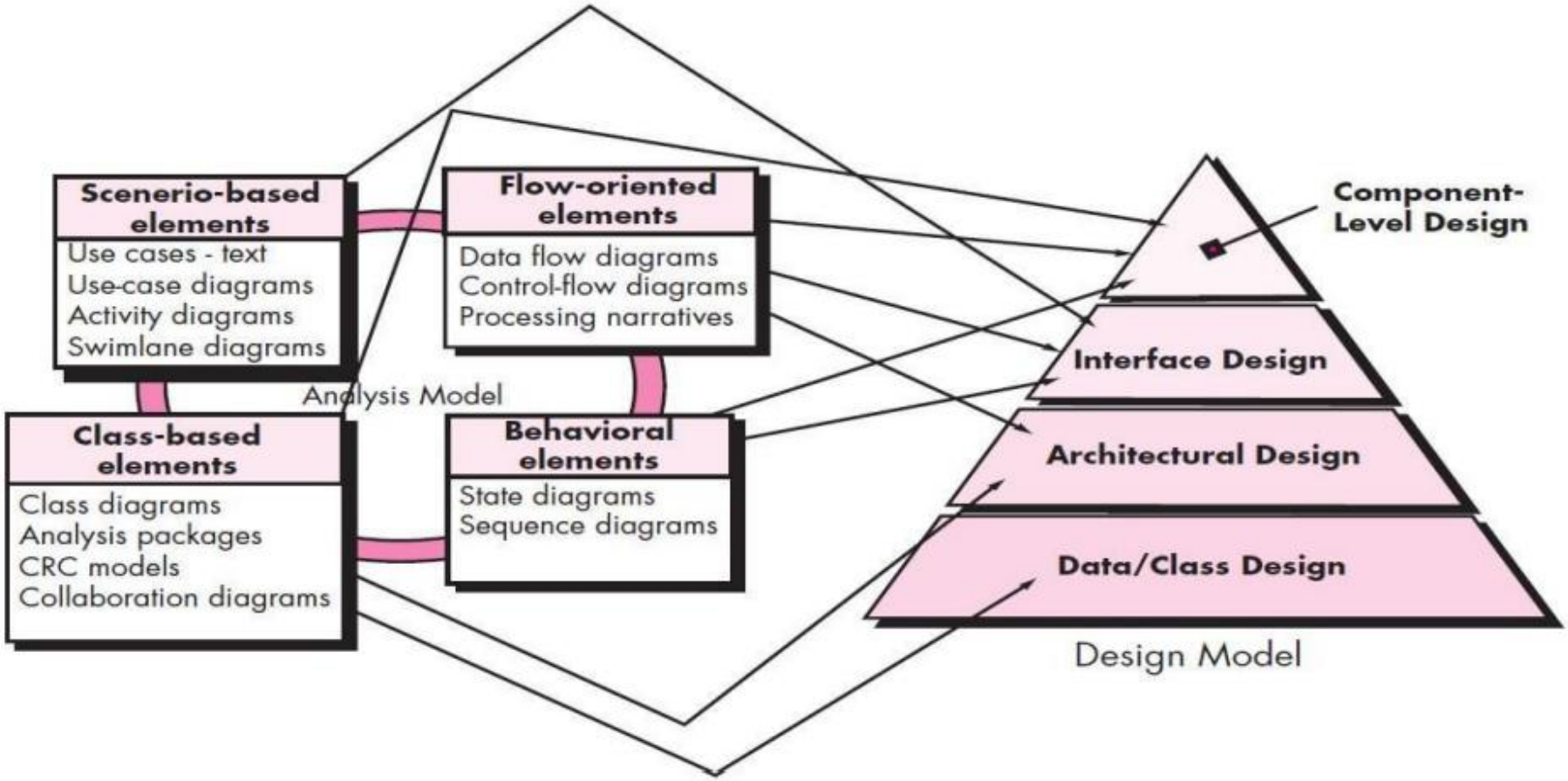
MSE: Types, diagrams

1. Architecture
2. User Interface



These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

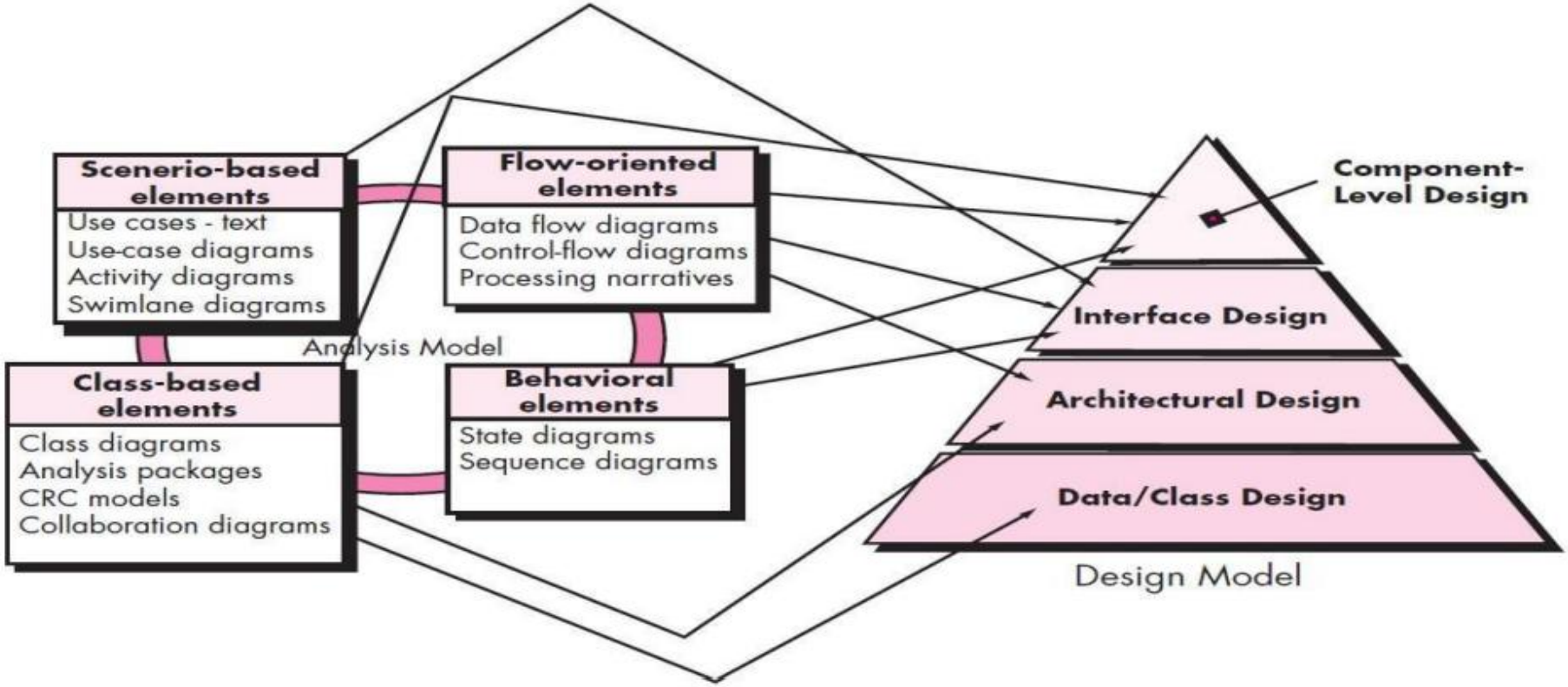
Fig : Translating the requirements model into the design



- ✓ The **data/class design** transforms class models into design class realizations and the requisite data structures required to implement the software.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

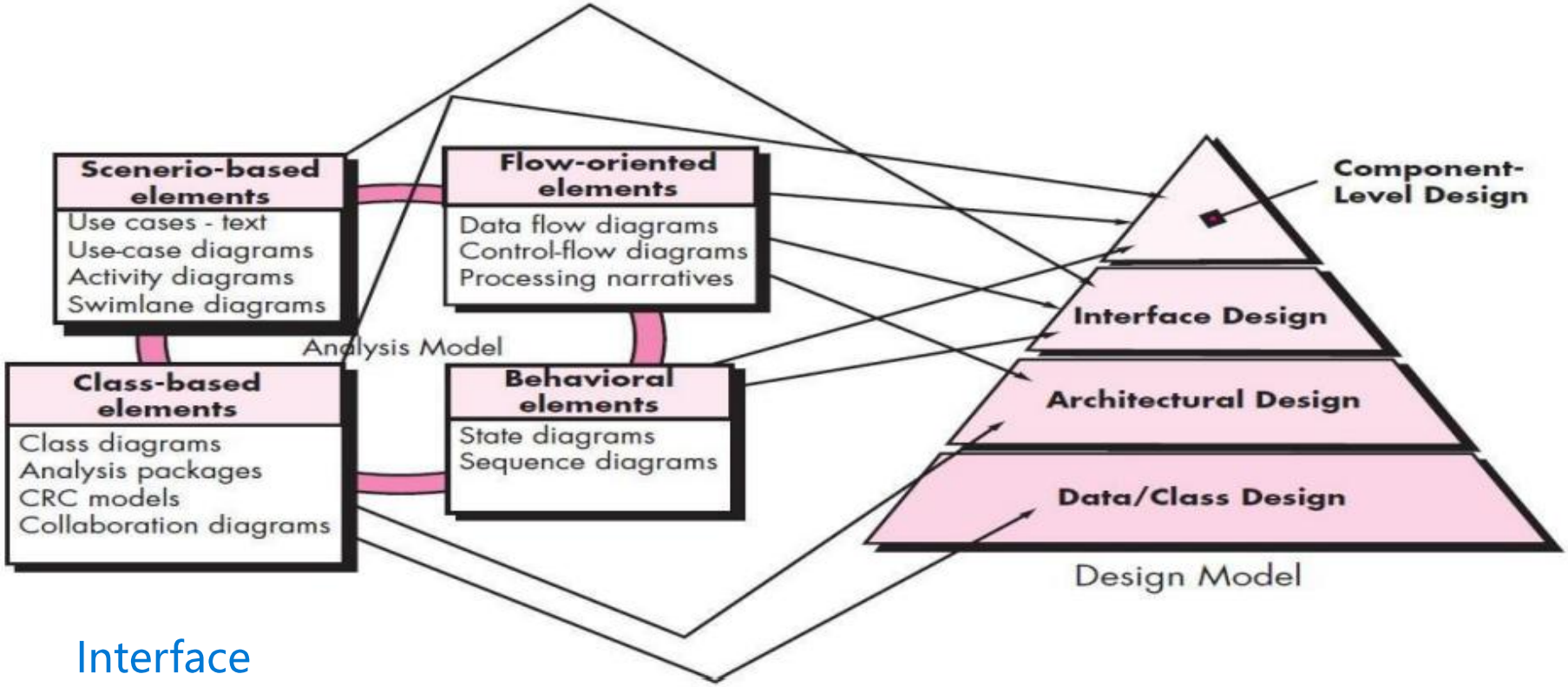
Fig : Translating the requirements model into the design



- ✓ The **architectural design** defines the relationship between major structural elements of the software, the architectural styles and design patterns that can be used to achieve the requirements defined for the system,

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Fig : Translating the requirements model into the design



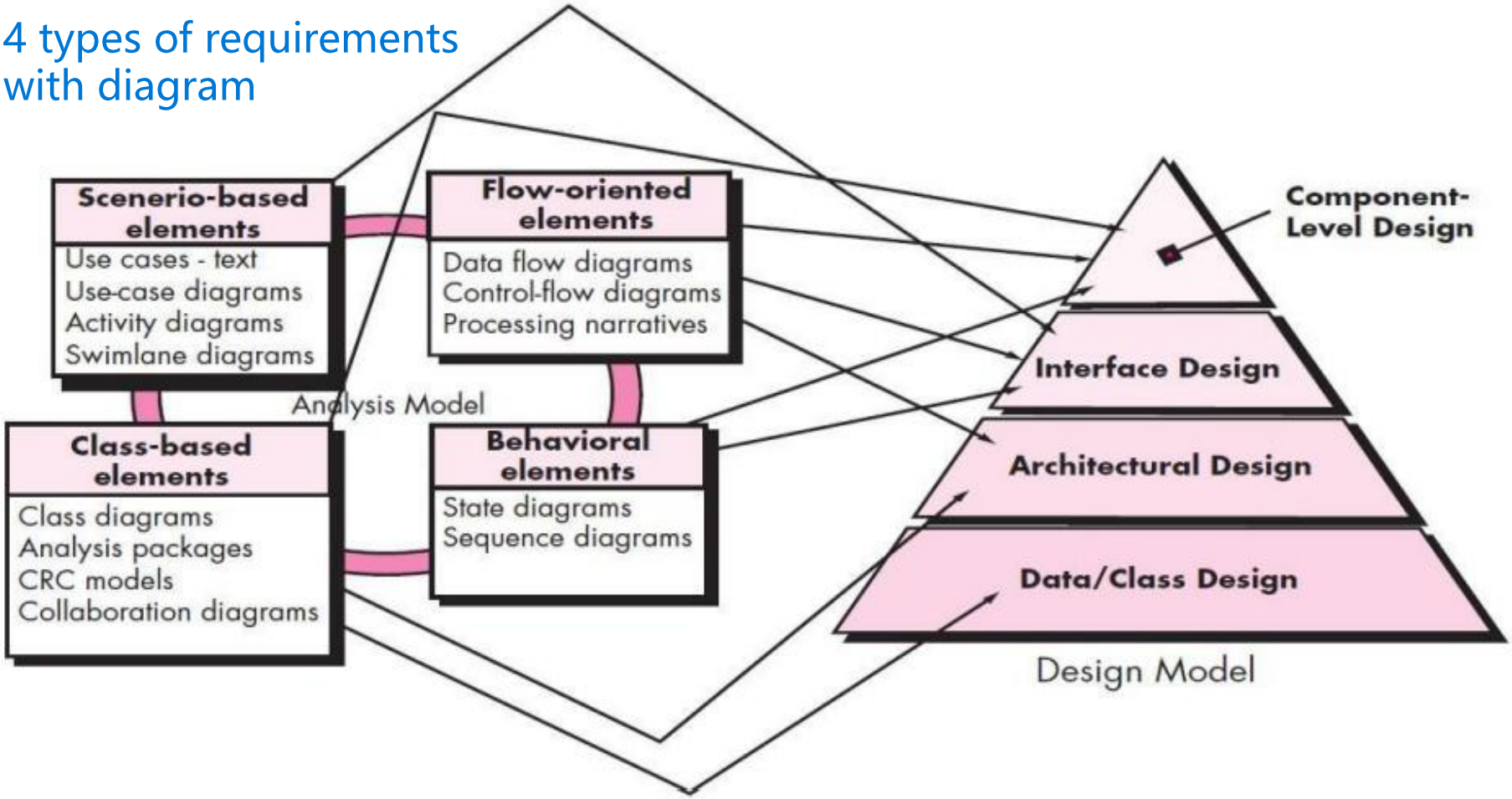
Interface

- ✓ The **interface design** describes how the software communicates with systems that interoperate with it, and with humans who use it. An interface implies a flow of information (e.g., data and/or control) and a specific type of behavior.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Fig : Translating the requirements model into the design

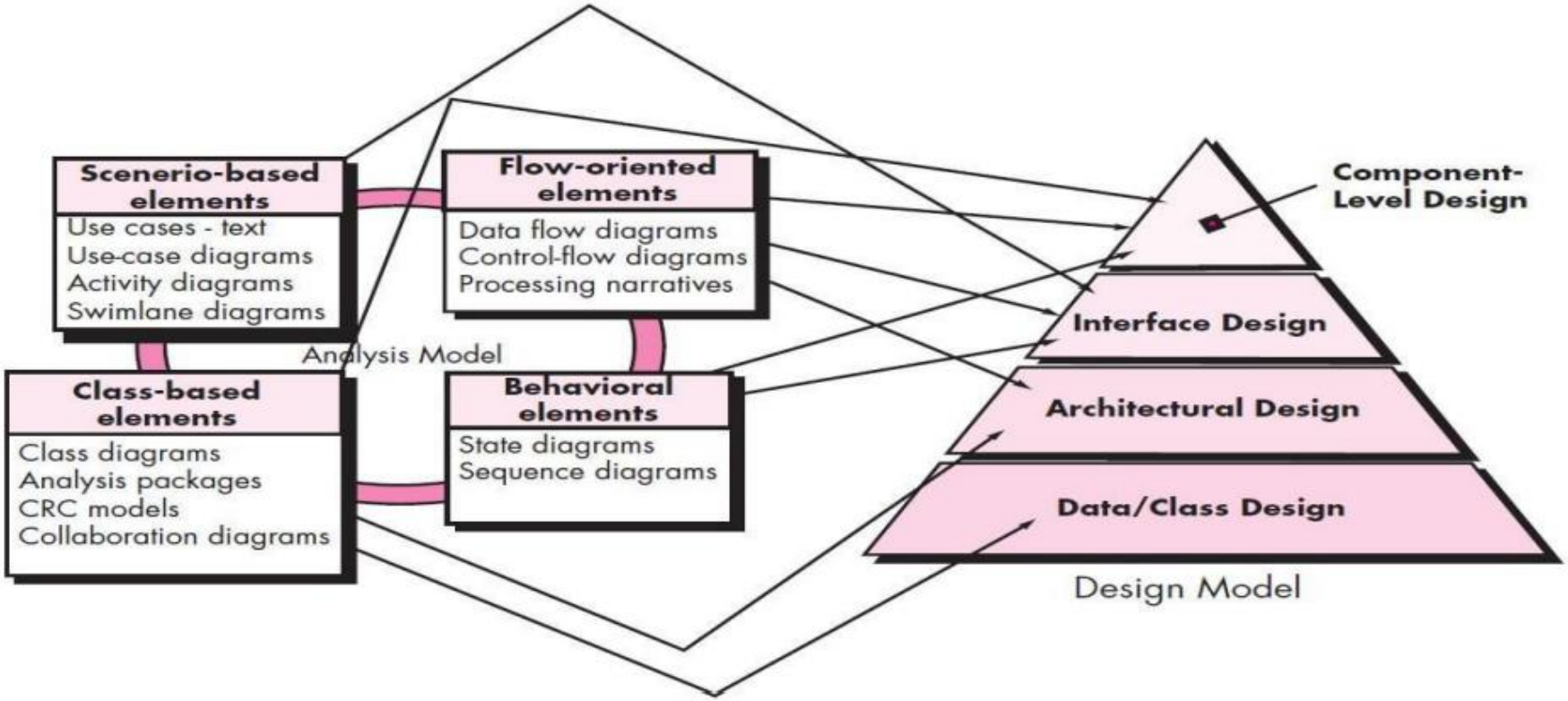
4 types of requirements with diagram



Therefore, usage scenarios and behavioral models provide much of the information required for interface design.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Fig : Translating the requirements model into the design



- ✓ The **component-level design** transforms structural elements of the software architecture into a procedural description of software components. Information obtained from the class-based models, flow models, and behavioral models serve as the basis for component design.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

DESIGN PROCESS :

- ✓ Software design is an iterative process through which requirements are translated into a "blueprint" for constructing the software.
- ✓ Initially, the blueprint depicts a holistic view of software. That is, the design is represented at a **high level of abstraction.**
- ✓ During the design process the software requirements model is transformed into design models that describe the details of the data structures, system architecture, interface and components.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design and Quality:

- ✓ The design must implement all of the explicit requirements contained in the analysis model, and it must accommodate all of the implicit requirements desired by the customer.
- ✓ The design must be a readable, understandable guide for those who generate code and for those who test and subsequently support the software.
- ✓ The design should provide a complete picture of the software, addressing the data, functional, and behavioral domains from an implementation perspective.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Quality Guidelines :

A design should exhibit an architecture that
(1) has been created using recognizable architectural styles or patterns,
(2) is composed of components that exhibit good design characteristics
(3) can be implemented in an evolutionary fashion
For smaller systems, design can sometimes be developed linearly.

- A design should be modular; that is, the software should be logically partitioned into elements or subsystems
- A design should contain distinct representations of data, architecture, interfaces, and components.
- A design should lead to data structures that are appropriate for the classes to be implemented and are drawn from recognizable data patterns.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

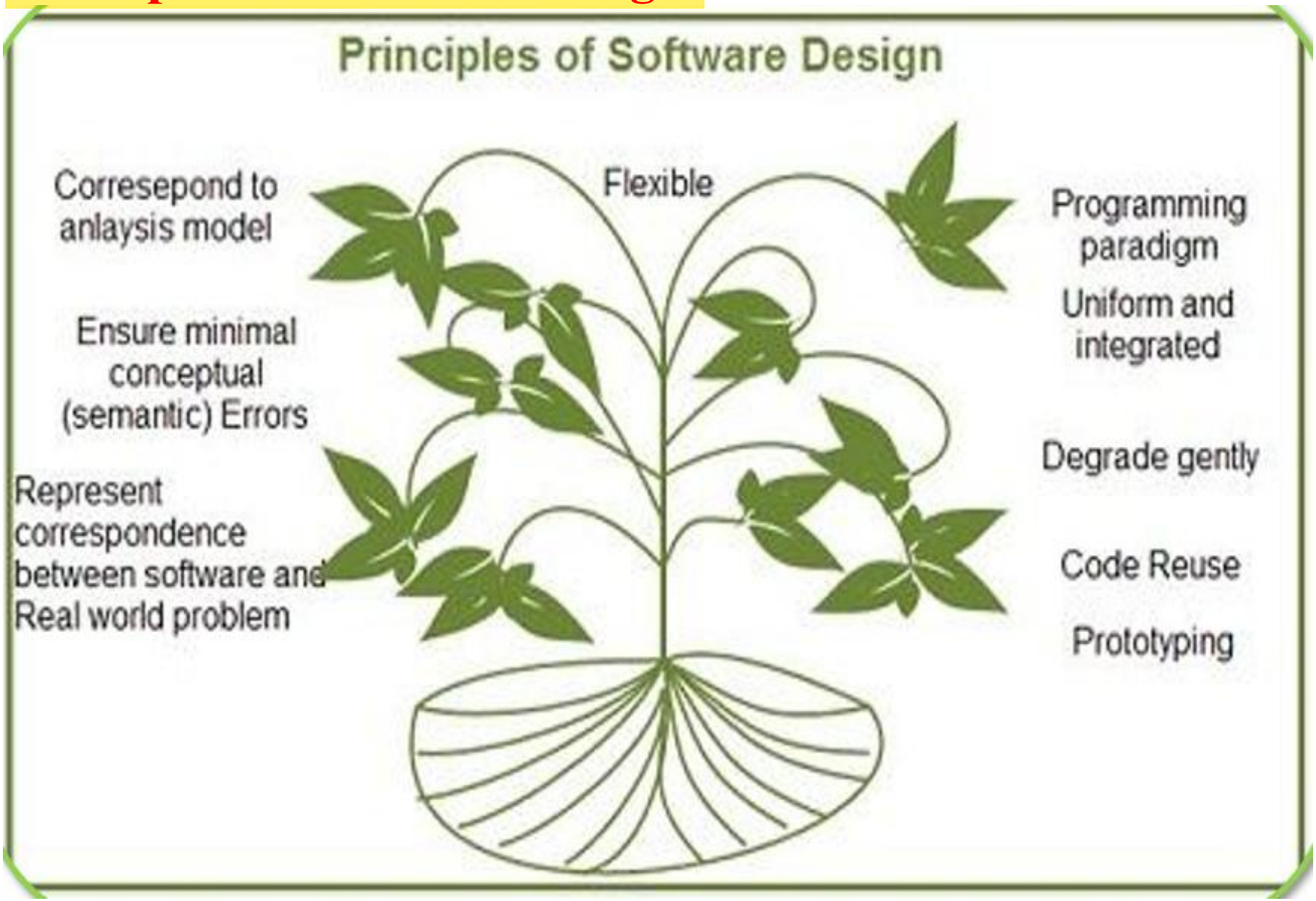
Quality Guidelines :

- A design should lead to components that exhibit independent functional characteristics.
- A design should lead to interfaces that reduce the complexity of connections between components and with the external environment.
- A design should be derived using a repeatable method that is driven by information obtained during software requirements analysis.
- A design should be represented using a notation that effectively communicates its meaning.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Principles of Software Design:

Imp MSE



These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Fundamental Design Concepts :

Imp MSE

1. **Abstraction**—data, procedure, control
2. **Architecture**—the overall structure of the software
3. **Patterns**—”conveys the essence” of a proven design solution
4. **Separation of concerns**—any complex problem can be more easily handled if it is subdivided into pieces
5. **Modularity**—compartmentalization of data and function
6. **Hiding**—controlled interfaces
7. **Functional independence**—single-minded function and low coupling
8. **Refinement**—elaboration of detail for all abstractions
9. **Aspects**—a mechanism for understanding how global requirements affect design
10. **Refactoring**—a reorganization technique that simplifies the design
11. **OO design concepts**— object-oriented (OO) paradigm.
12. **Design Classes**—provide design detail that will enable analysis classes to be implemented

These slides are designed to accompany Software Engineering: A Practitioner’s Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 1) Abstraction

Abstraction allows designers to focus on solving a problem without being concerned about irrelevant lower-level details.

There are two types of abstraction available :

Procedural abstraction – a sequence of instructions that have a specific and limited function.

Ex: Word OPEN for a door.

Data abstraction – a named collection of data that describes a data object. Data abstraction for door would be a set of attributes that describes the door (e.g. door type, swing direction, weight, dimension).

Control abstraction – a program control mechanism without specifying internal detail. It is used to coordinate all activities in operating system

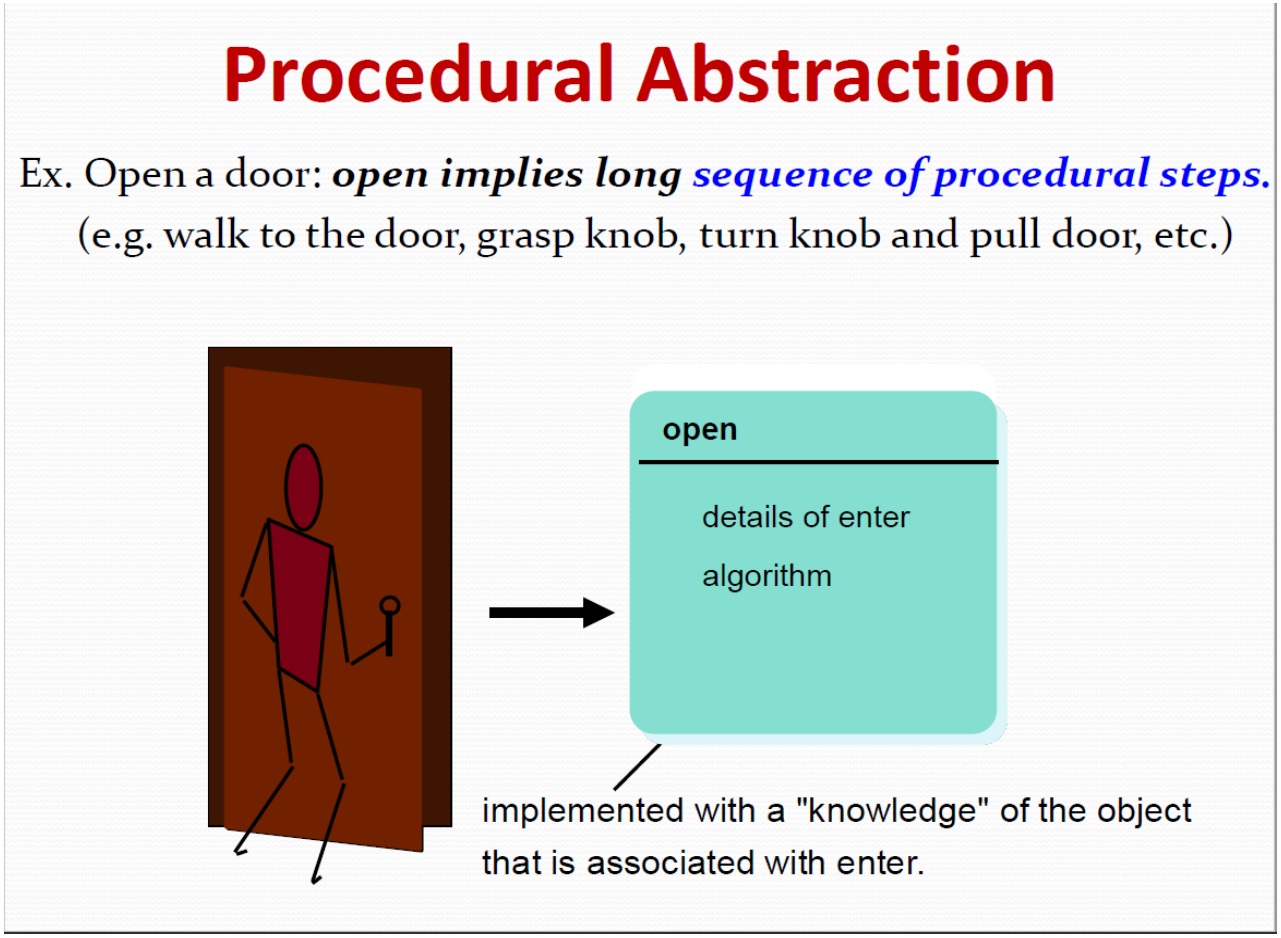
These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 1) Abstraction

Procedural abstraction – a sequence of instructions that have a specific and limited function.

Ex: Word OPEN for a door.

Open implies a Long sequence of Procedural steps – walk to the door, reach out and grasp the knob, turn knob and pull the door, step away from the door, etc….



These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 1) Abstraction

Data abstraction is a named collection of data that describes the data object.

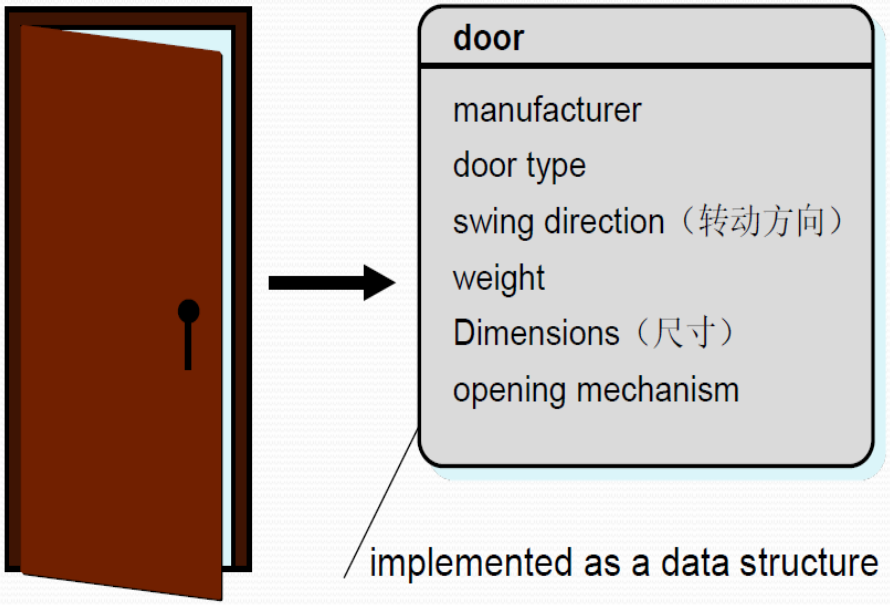
data abstraction for **door** would Encompass a set of attributes that describe the door. **Door type, swing direction, opening mechanism, weight, dimensions.**

.

Data Abstraction

Ex. Open a door. – *door is data object*.

Data abstraction for door would encompass a set of attributes describing the door. (E.g. door type, swing direction, opening mechanism, etc.)



These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 2) Architecture

- Software architecture refers to the overall structure of software and the ways in which that structure provides conceptual integrity for a system.
- Architecture is the structure or organization of program components (modules), the manner in which these components interact and the structure of data that are used by the components.
- The architecture design can be represented using one or more number of different models.

Architecture Models:

- Structural models represent architecture as an organized **collection of program components.**

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 2) Architecture

Architecture Models:

- Framework models increase the level of design abstraction by attempting to identify **repeatable architecture design frameworks** that are encountered in similar types of applications.
- Dynamic models address the **behavioral aspects of the program architecture,** indicating how the structure or system configuration may change as a function of external events.
- Process models focus on the **design of the business or technical process** that the system must accommodate.
- Functional models can be used to represent the **functional hierarchy of the system.**

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 3) Patterns

A design pattern in software engineering is a **reusable solution to a commonly occurring problem** within a given context in software design.

It is not a finished design that can be directly transformed into code but a **template for how to solve a problem that can be used in many different situations**.

Design patterns are best practices that programmers can use **to solve common problems when designing an application or system.**

The intent of each design pattern is to provide a description that enables a designer to determine: 1)Whether the pattern is applicable to the current work. 2) Whether the pattern can be reused. 3)Whether the pattern can serve as a guide for developing a similar, but functionally or structurally different pattern.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 3) Patterns

Types of Design Patterns:

Architectural patterns: These patterns are high-level strategies that refer to the overall structure and organization of a software system. That is, they define the elements of a software system such as subsystems, components, classes, etc.

Design patterns: These patterns are medium-level strategies that are used to solve design problems. They provide a means for the refinement of the elements of a software system or the relationship among components or mechanisms that affect component-to-component.

Idioms: These patterns are low-level patterns, which are programming-language specific. They describe the implementation of a software component, the method used for interaction among software components.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 4) Separation of Concerns:

- Any complex problem can be more easily handled if it is subdivided into pieces that can each be solved and/or optimized independently
- A concern is a **feature or behavior that is specified as part of the requirements model** for the software
- By separating concerns into smaller, and therefore more manageable pieces, a problem takes less effort and time to solve.
- This leads to **divide-and- conquer** strategy:
  it's easier to solve a complex problem when you break it into manageable pieces.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 4) Separation of Concerns:

- Example:
  In a web application, you might separate concerns by having a
  model layer (for business logic anddata),
  controller layer (for handling requests), and a
  view layer (for user interface rendering).
  Each layer has a distinct responsibility.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 5) Modularity:

- Definition: **Modularity** refers to the design technique where a software system is divided into discrete, self-contained modules, each responsible for a part of the functionality. A module could be a package, function, or class that encapsulates specific functionality.

- Purpose: Modularity aims to make the software system more maintainable, reusable, and scalable by breaking it into independent, interchangeable components.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 5) Modularity:

- Purpose: Each module should have a well-defined interface and hide its internal implementation details.

- Example: A library of utility functions that handle date manipulation, string operations, or math calculations would be a modular design, where each function or set of functions is encapsulated in its own module.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 6) Information Hiding:

- ✓ This is one of the important property of an effective modular design.
- ✓ The term information hiding means the modules are designed in such a way that **information contained in one module cannot be accessible to other module.**
- ✓ Due to information hiding **only limited amount of information can be passed to other module or any local data structure used by other module.**
- ✓ **The main advantage lies basically in testing and maintenance.**
- ✓ Due to information hiding **some data and procedures of one module can be hidden from another module.**
- ✓ This ultimately **avoids introduction of errors from one module to another.**

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 6) Information Hiding:

- ✓ Similarly, **one can make changes in the desired module without affecting the other module.**

- ✓ **Example:**
- ✓ When designing an API, you provide external consumers with a set of functions or endpoints they can interact with.
- ✓ The internal logic (how data is processed, how services communicate) is hidden behind the API.
- ✓ This allows the API provider to change the implementation without breaking the consumer's code.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 7) Functional Independence :

- ✓ Functional independence is achieved by developing modules with "single-minded" function and an "aversion" to excessive interaction with other modules.

- ✓ Independence is measured using two qualitative criteria: Criteria 1) **Cohesion and** Criteria 1) **Coupling**

- ✓ **Cohesion** is an indication of the relative functional strength of a module.
- ✓ A cohesive module performs a single task, requiring little interaction with other components in other parts of a program. Stated simply, a cohesive module should (ideally) do just one thing.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 7) Functional Independence :

- ✓ The degree in which module perform functions in inner module in the system.
- ✓ <u>High cohesion</u> necessary in good software

- ✓ **Coupling** is an indication of the relative interdependence among modules.
- ✓ Coupling depends on the interface complexity between modules, the point at which entry or reference is made to a module, and what data pass across the interface.
- ✓ The degree in which module is "connected" to other module in the system.
- ✓ <u>Low coupling</u> necessary in good software

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 7) Refinement :

- Stepwise refinement is a **top-down design strategy**.
- Refinement is actually **a process of elaboration**.
- Begin with a statement of function (or description of information) that is defined at a high level of abstraction. The statement describes function or information conceptually but provides no information about the internal workings of the function or the internal structure of the information.
- Then provide more and more detail as each successive refinement (elaboration) occurs.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 7) Refinement :

- **Abstraction** and **refinement** are complementary concepts.
- Abstraction enables a designer to specify procedure and data internally and but suppress low-level details for outsiders.
- Refinement helps the designer to expose low-level details as design progresses.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 8) Aspects :

- An *aspect* is a representation of a cross-cutting concern.
- Consider two requirements, *A* and *B*. Requirement *A* *crosscuts* requirement *B* "if a software refinement has been chosen in which *B* cannot be satisfied without taking *A* into account.
- **Examples of Aspects in Software Architecture:**
  - Security
  - Logging and Auditing
  - Error Handling
  - Performance Monitoring
  - Transactions Management

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 9) Aspects :

- An *aspect* is a representation of a cross-cutting concern.
- Consider two requirements, *A* and *B*. Requirement *A* *crosscuts* requirement *B* "if a software refinement has been chosen in which *B* cannot be satisfied without taking *A* into account.
- **Examples of Aspects in Software Architecture:**
  - Security
  - Logging and Auditing
  - Error Handling
  - Performance Monitoring
  - Transactions Management

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 10) Refactoring :

- Restructuring (rearranging) code…
- "changing the structure of a program so that the functionality is not changed"
- Refactoring is a reorganization technique that simplifies the design (or internal code structure) of a component without changing its function or external behaviour.
- It removes redundancy, unused design elements, inefficient or unnecessary algorithms, poorly constructed or inappropriate data structures, or any other design failures.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 11) Refactoring :

- Restructuring (rearranging) code…
- "changing the structure of a program so that the functionality is not changed"
- Refactoring is a reorganization technique that simplifies the design (or internal code structure) of a component without changing its function or external behaviour.
- It removes redundancy, unused design elements, inefficient or unnecessary algorithms, poorly constructed or inappropriate data structures, or any other design failures.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 12) Object Oriented (OO) Design Concepts :

- "Design system using self-contained objects and object classes"
- Objects are abstractions of real world or system entities and manage themselves
- Objects are independent and encapsulate state and represent information
- System functionality is expressed in terms of object services
- Shared data are eliminated. Objects communicate by message passing

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 13) Design Classes :

Design classes are defined as the classes that describe some elements of problem domain in which the problem is viewed from users' point of view.

The goal of design classes is:

- Analysis classes are refined during design to become entity classes
- Boundary classes are developed during design to create the interface that the user sees and interacts with as the software is used.
- Boundary classes are designed with the responsibility of managing the way entity objects are represented to users.

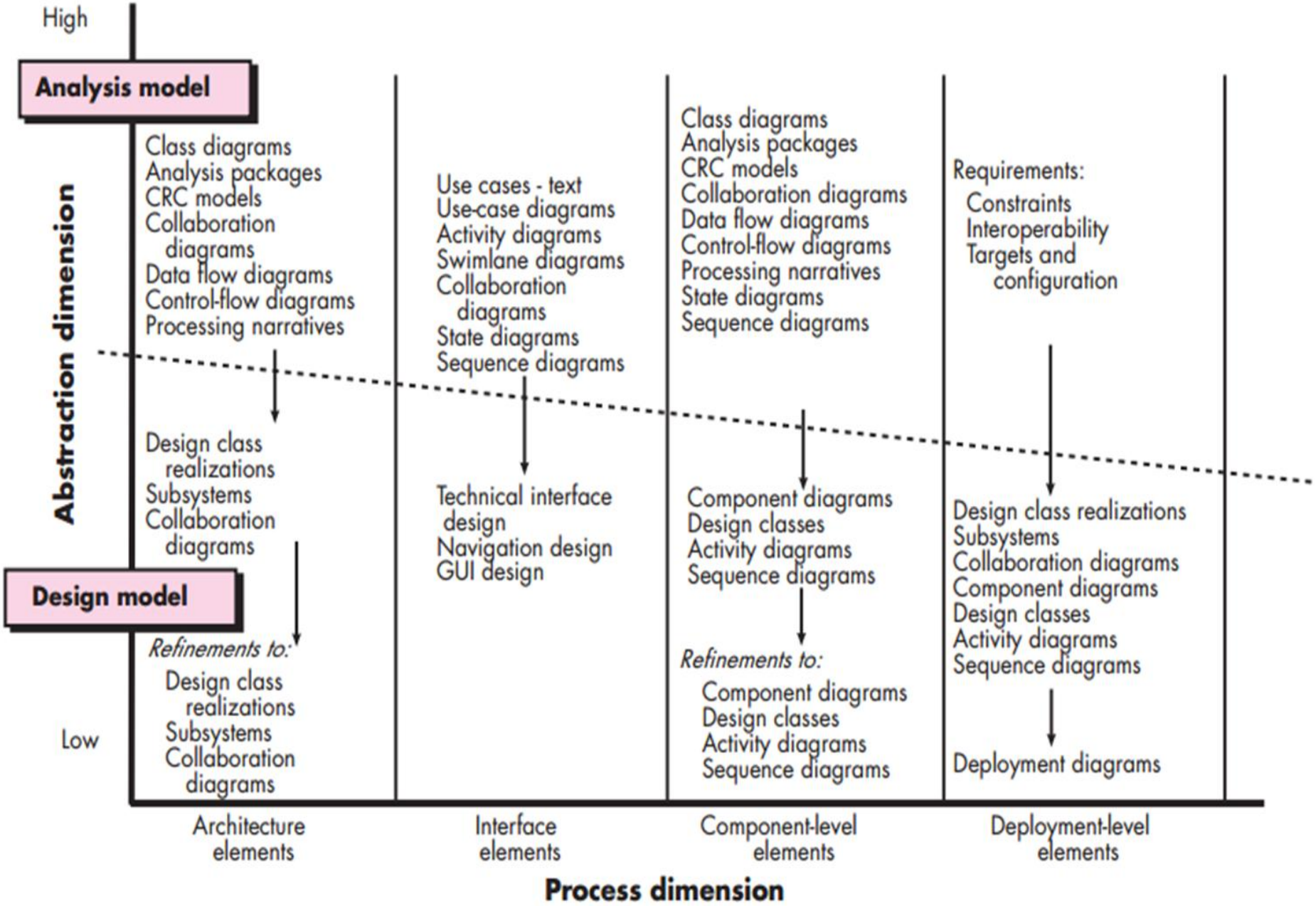
These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Concepts : 13) Design Classes :

- Controller classes are designed to manage the creation or update of entity objects;
  - the instantiation of boundary objects as they obtain information from entity objects;
  - complex communication between sets of objects;
  - validation of data communicated between objects or between the user and the application.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Design Models :



Design Model Elements :

Data elements

Data model --> data structures

Data model --> database architecture

It produced a model of data that represent a high level of abstraction

Architectural elements

Application domain

Analysis classes, their relationships, collaborations and behaviors are transformed into design realizations Patterns and "styles"

Interface elements

The user interface (UI) external interfaces to other systems, devices, networks or other producers or consumers of information internal interfaces between various design components.

Component elements The component-level design for software fully describes the internal detail of each software component.

Deployment elements Deployment-level design elements indicate how software functionality and subsystems will be allocated within the physical computing environment that will support the software.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Architectural Design:

Architecture?

The architecture is not the operational software. Rather, it is a representation that enables a software engineer to:

(1) analyze the effectiveness of the design in meeting its stated requirements,

(2) consider architectural alternatives at a stage when making design changes is still relatively easy, and

(3) reduce the risks associated with the construction of the software.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Architectural Design:

Why is Architecture Important?

- Representations of software architecture are an enabler for communication between all parties (stakeholders) interested in the development of a computer-based system.
- The architecture highlights early design decisions that will have a profound impact on all software engineering work that follows and, as important, on the ultimate success of the system as an operational entity.
- Architecture "constitutes a relatively small, intellectually graspable mode of how the system is structured and how its components work together"

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Architectural Design:

- Architectural design involves three main tasks:

✓ **Represent the system in context**

➢ Define the external entities (other systems, devices, people) that the software interacts with

➢ Use Architectural Context Diagram (ACD)

✓ **Define Archetypes**

➢ An archetype is an abstraction (similar to class) that represents one element of system behaviour

➢ Four Architectural archetype should be identified

- **Node, Detector, Indicator, Controller**

➢ Use UML

✓ **Refine the Architecture into components**

➢ The designer specifies the structure of the system by defining and refining software components that implement each archetype

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Architectural Design:

Representing the System in Context:

At the architectural design level, a software architect uses an architectural context diagram(ACD) to model the way software interacts with entities external to its boundaries.

Referring to the figure, systems that interoperate with the target system (system for which an architectural design is to be developed) are represented as

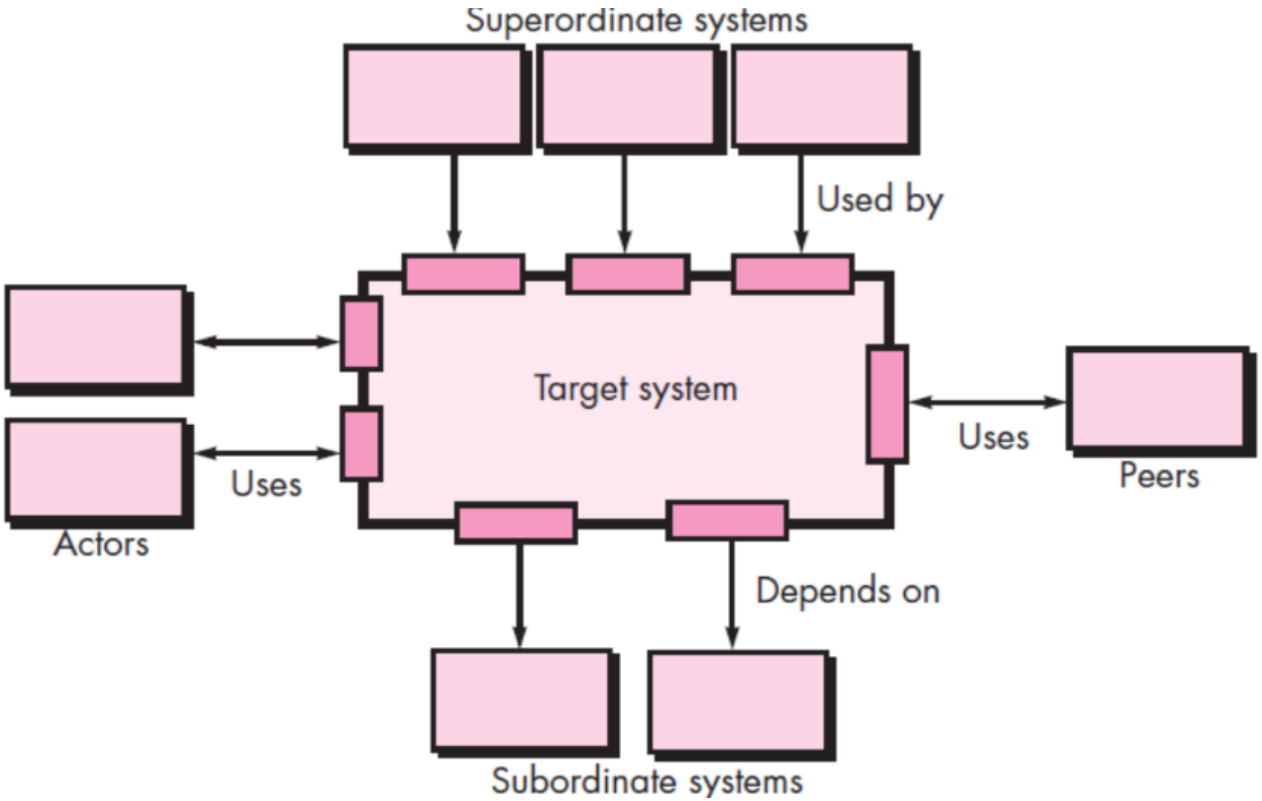


Fig : Architectural context diagram

Architectural Design:

Representing the System in Context:

- Superordinate systems—those systems that use the target system as part of some higher-level processing scheme.

Subordinate systems—those systems that are used by the target system & Provide data or processing that are necessary to complete target system functionality.

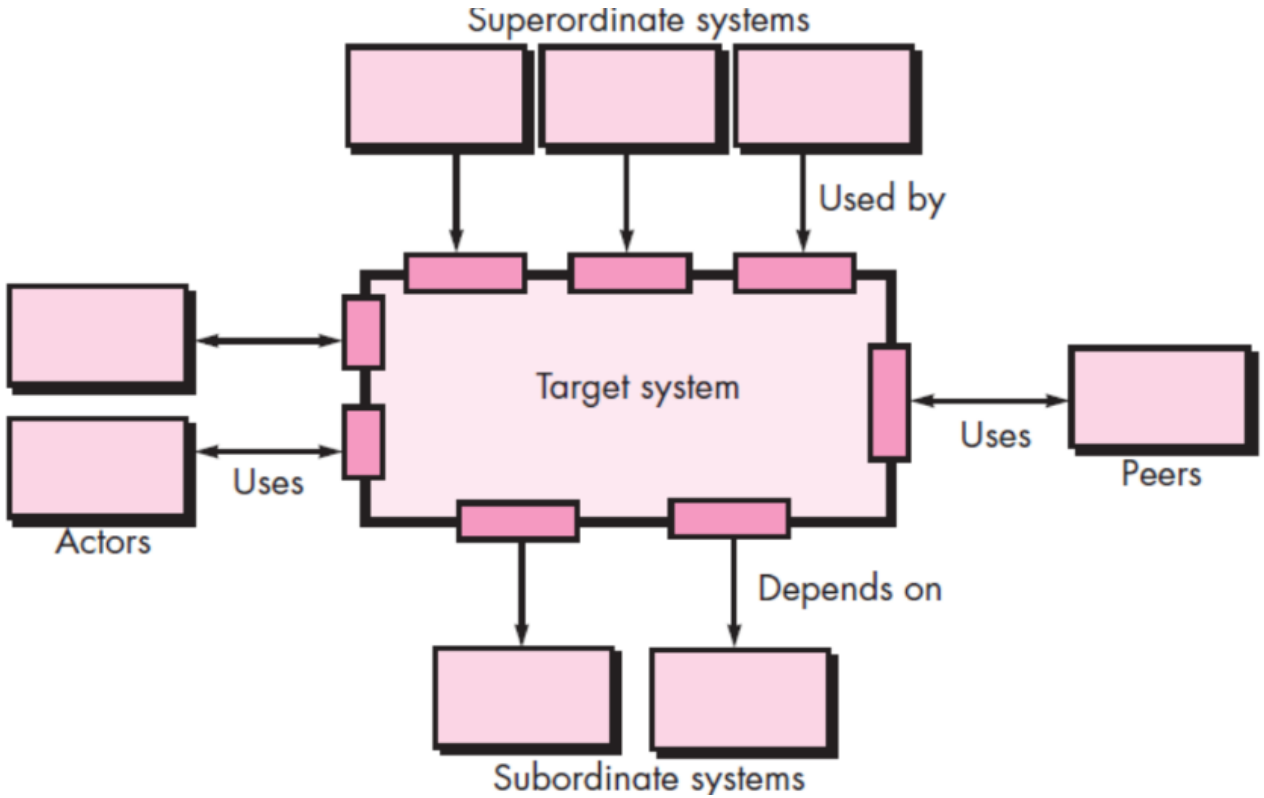


Fig : Architectural context diagram

Architectural Design:

Representing the System in Context:

- Peer-level systems - those systems that interact on a peer-to-peer basis (i.e., information is either produced or consumed by the peers and the target system.
- Actors- entities (people, devices) that interact with the target system by producing or consuming information that is necessary for requisite processing.

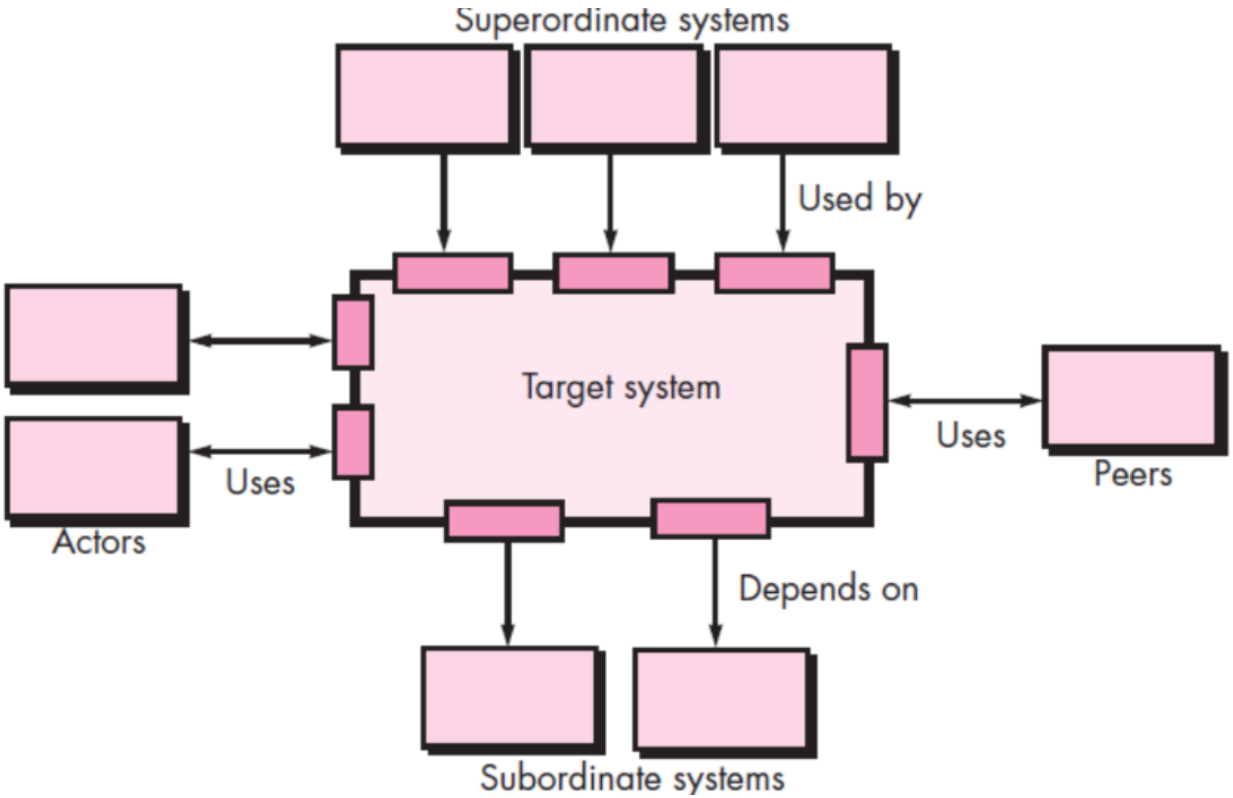


Fig : Architectural context diagram

Architectural Design:

Archetypes:

- ✓ In software architecture design, archetypes represent basic, high-level building blocks or abstract models that define the fundamental structure of a system.
- ✓ These are recurring patterns or common components that appear in various types of systems.
- ✓ Archetypes serve as a blueprint or conceptual framework for designing more specific solutions in software architecture.
- ✓ Archetypes help define the core elements of a system, their responsibilities, and how they interact with one another. They can be thought of as foundational templates or generic building blocks that form the basis for different system designs.

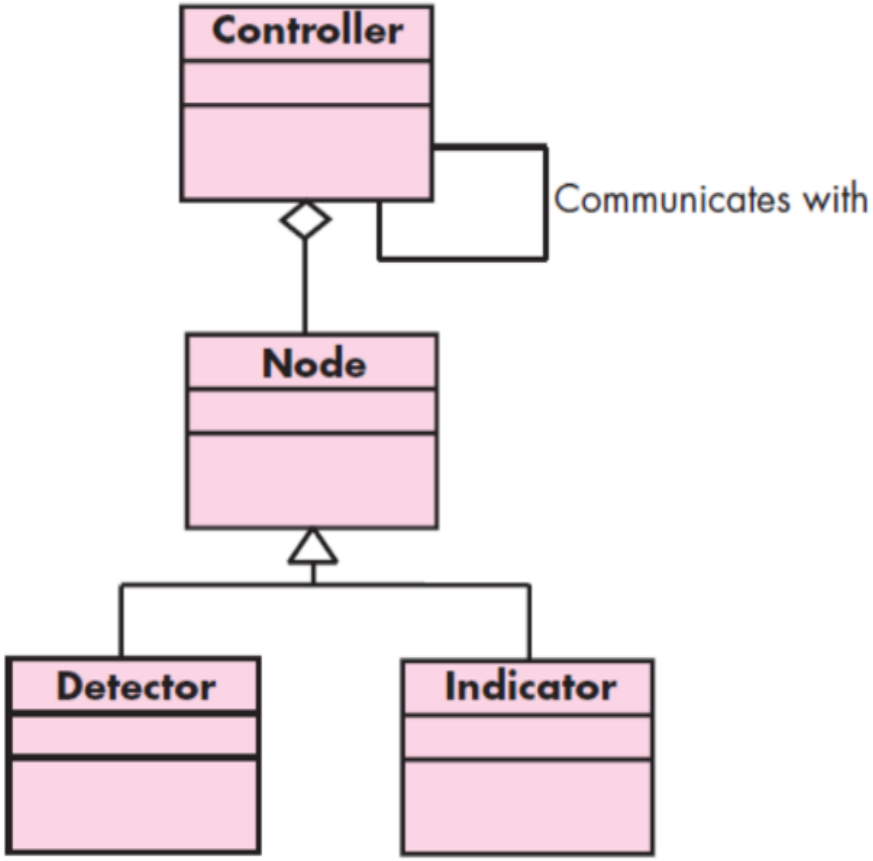
Architectural Design:

Archetypes: The following archetypes can be used :

Node: Represents a cohesive collection of input and output elements of the home security function.

Example: a node might be comprised of (1) various sensors (2) a variety of alarm (output) indicators.

Detector: An abstraction that encompasses all sensing equipment that feeds Information into the target system.

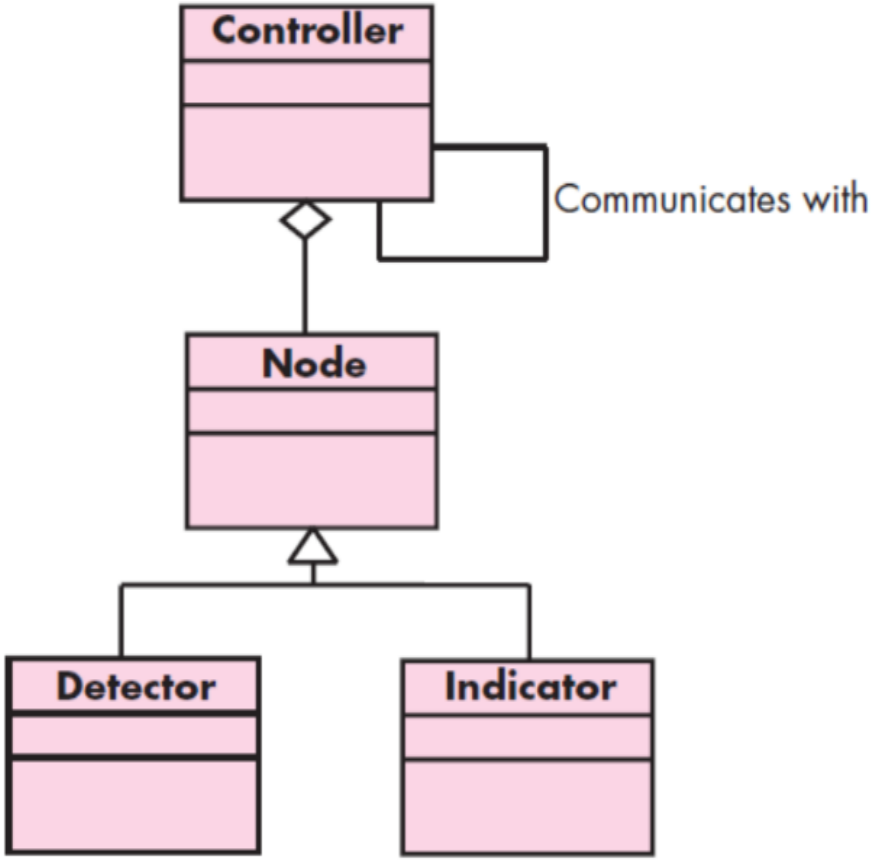


Architectural Design:

Archetypes: The following archetypes can be used :

Indicator: An abstraction that represents all mechanisms (e.g., alarm siren, flashing lights, bell) for indicating that an alarm condition is occurring.

Controller: An abstraction that depicts the mechanism that allows the arming or disarming of a node. If controllers reside on a network, they have the ability to communicate with one another.



Architectural Design:

Refining the Architecture into Components:

As the software architecture is refined into components, the structure of the system begins to emerge.

The architecture must accommodate many infrastructure components that enable application components but have no business connection to the application domain.

Set of top-level components that address the following functionality:

- External communication management –

coordinates communication of the security function with external entities such as other Internet-based systems and external alarm
notification.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Architectural Design:

Refining the Architecture into Components:

Control panel processing

manages all control panel functionality.

Detector management

coordinates access to all detectors attached to the system.

Alarm processing

verifies and acts on all alarm conditions.

Each of these top-level components would have to be elaborated iteratively and then positioned within the overall architecture.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

User Interface Design Model:

User Interface is the front-end application view to which user interacts with the software.
It determines how commands are given to the computer or the program and how data is displayed on the screen.

The software becomes more popular if its user interface is:

1. Attractive
2. Simple to use
3. Responsive in short time
4. Clear to understand
5. Consistent on all interfacing screens.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Types of User Interface:

Type 1 : Text-Based User Interface or Command Line Interface

- Text-Based User Interface primarily used Keyboard handling data.
- Command Line Interface provide command propomt or coding tools where the user types the command towards the system.
- Command Line Interface used by Technical people or Programmers.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

Types of User Interface:

Type 2 : Graphical User Interface

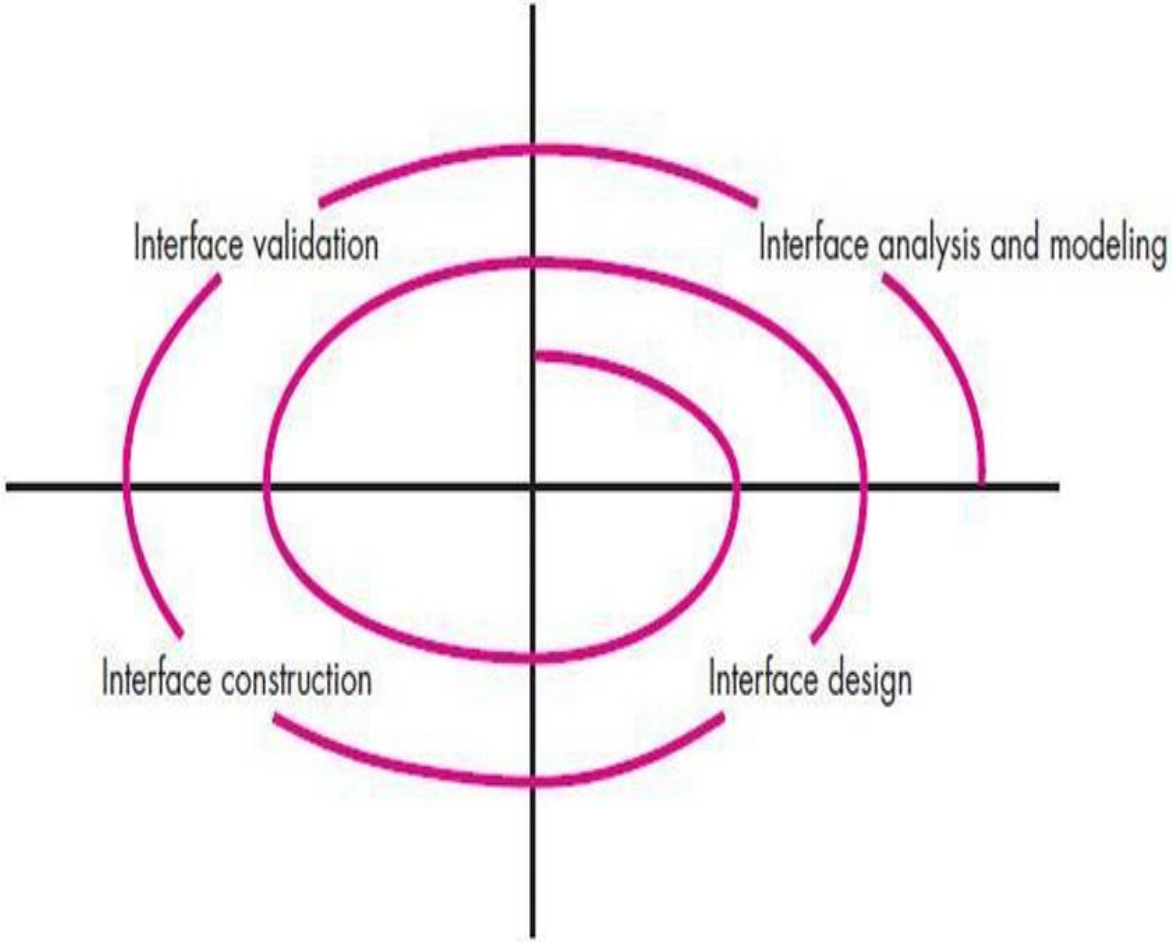
- Graphical User Interface provides the simple interactive Interface to interact with the system.
- GUI can be a combination of both hardware and software.
- Graphical User Interface are easy to learn as compared to the command Line Interface.
- GUI provides multiple windows to the user simultaneously to interact with the system.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

User Interface Design Model :

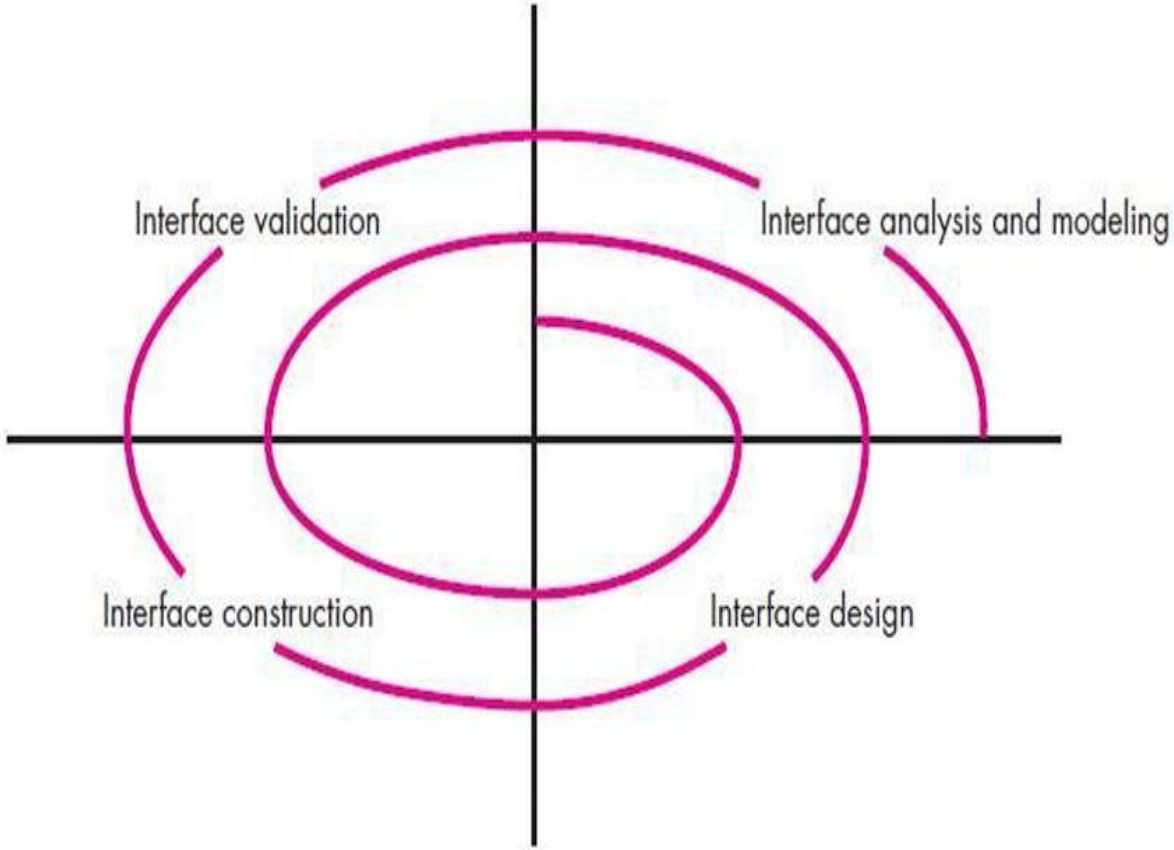
The goal of **interface design** is to define a set of interface objects and actions that enable a user to perform all defined tasks in a manner that meets every usability goal defined for the system.

Fig : The user interface design process



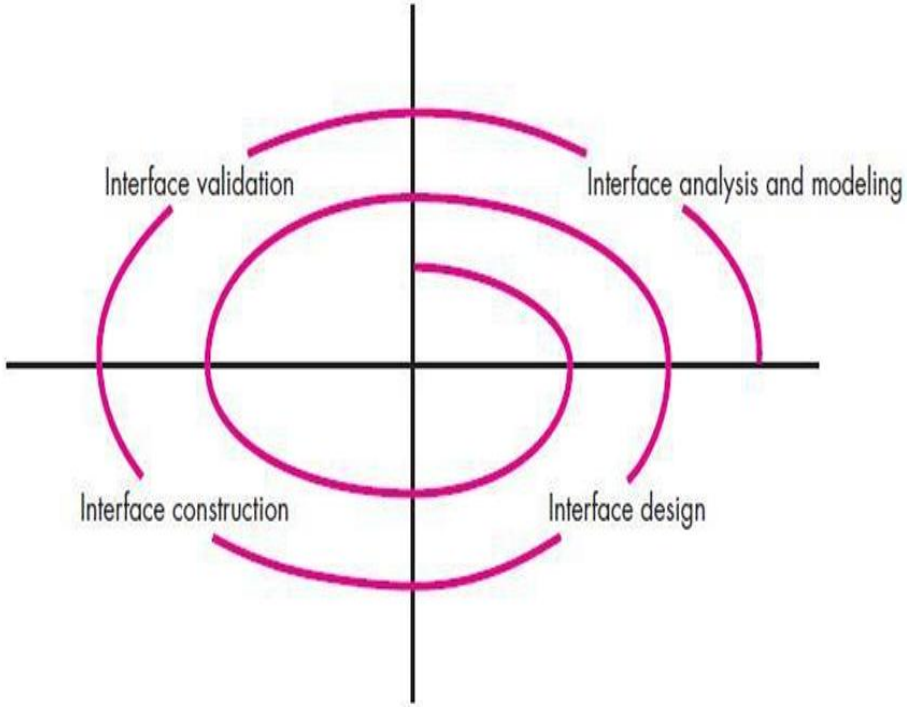
User Interface Design Model :

Interface construction normally begins with the creation of a prototype that enables usage scenarios to be evaluated. As the iterative design process continues, a user interface tool kit may be used to complete the construction of the interface.



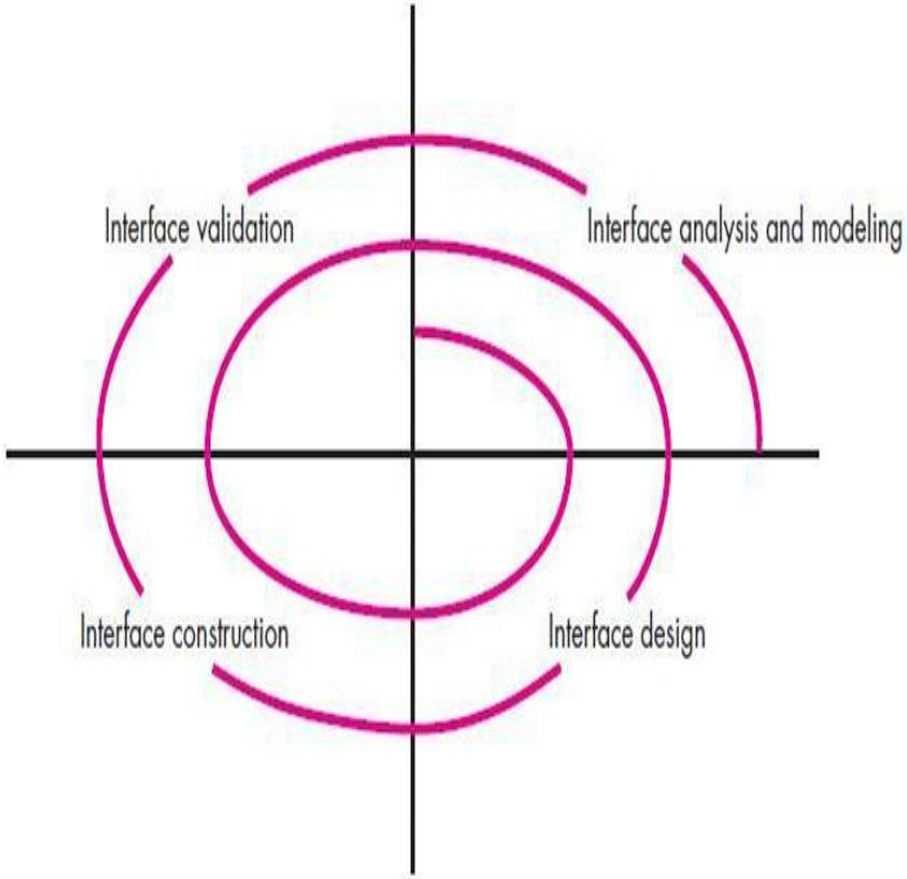
User Interface Design Model :

Interface validation focuses on (1) the ability of the interface to implement every user task correctly, to accommodate all task variations, and to achieve all general user requirements; (2) the degree to which the interface is easy to use and easy to learn, and (3) the users' acceptance of the interface as a useful tool in their work.



User Interface Design Model :

Interface Analysis focuses on understanding the problem means understanding (1) the people (end users) who will interact with the system through the interface, (2) the tasks that end users must perform to do their work, (3) the content that is presented as part of the interface, and (4) the environment in which these tasks will be conducted.



User Interface Design Model :

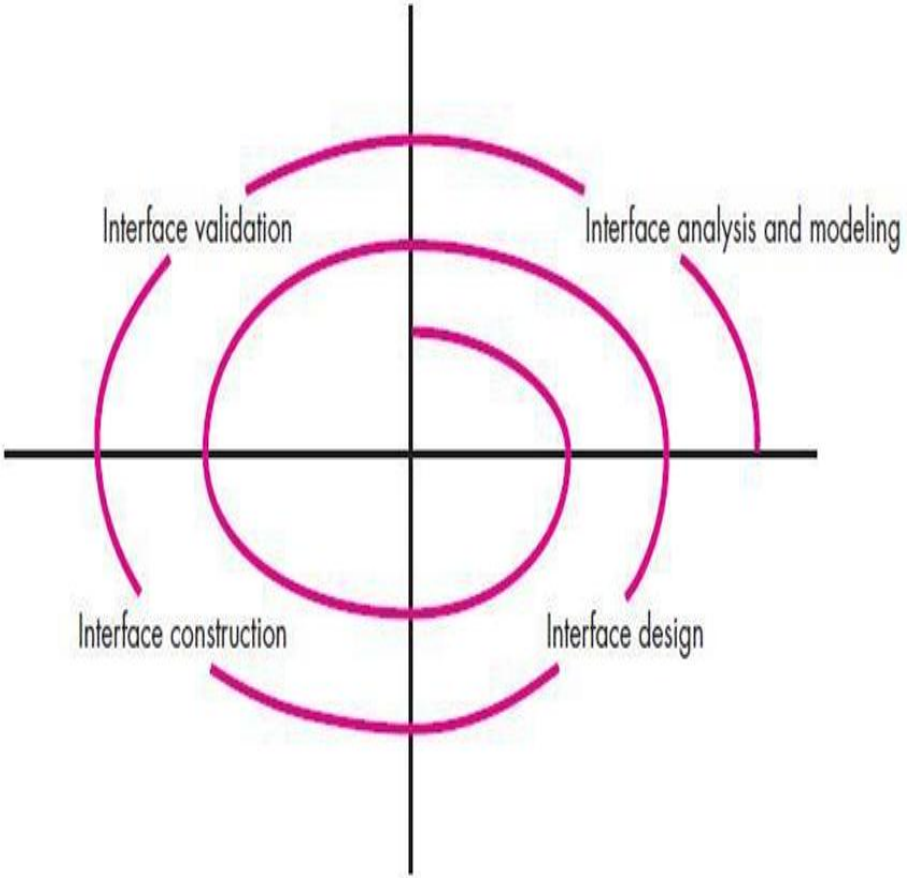
Task Analysis and Modeling goal is to answer the following questions: • What work will the user perform in specific circumstances?

• What tasks and subtasks will be performed as the user does the work?

• What specific problem domain objects will the user manipulate as work is performed?

• What is the sequence of work tasks—the workflow?

• What is the hierarchy of tasks?



User Interface Design Principles :

1. User Familiarity: The interface should be based of user-oriented terms & concepts.
   Example: In windows OS, Terms like Desktop, Document, Folder, Rename, Power etc.
2. Consistency: The system command and menus should have the same format & parameter.
   Example: Group of fonts, sizes, colors etc.
3. Minimal Surprise: User never like to see the system working in an unexpected manner.
   They get frustrated in such cases. Develop as per the user tendency.
   Example: Group of similar type of features, In MS Words, File-> New Open, Save, Print, Close, Restart System.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

User Interface Design Golden Rules :

1. Place the user in control.

- ✓ The User should be able to easily enter and exit the mode.
- ✓ Provide flexible interaction using keyboard, mouse or touch screen etc.
- ✓ User must be able to interrupt the sequence to do some other work.
- ✓ Display descriptive messages and text
- ✓ Allow users to customize the interface like short cut keys etc.
- ✓ Hide technical internal details from users.
- ✓ Allow users to directly manipulate interface objects.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

User Interface Design Golden Rules :

2. Reduce User's Memory Load

- ✓ Reduce demand on short term memory
- ✓ New features updated in system.
- ✓ Define shortcuts that are intuitive.
- ✓ The visual layout of the interface should be based on a real world metaphor.
- ✓ Disclose information in a hierarchical & progressive fashion.

3. Make the interface Consistent

- ✓ Meaningful user tasks performed.
- ✓ Maintain consistency across a family of applications.
- ✓ Keep interaction results the same as per user expectations.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

User Interface Analysis and Design Models :

Four different models come into play when a user interface is to be analyzed and designed.

1. A human engineer (or the software engineer) establishes a user model,
2. the software engineer creates a **design model**,
3. the end user develops a mental image that is often called the **system perception**,
4. and the implementers of the system create an **implementation model.**

These models enable the interface designer to satisfy a key element of the most important principle of user interface design: "**Know the user, know the tasks**."

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.

User Interface Design Issues:

1. Response Time: Time between request and response of the system. If the response time is too long, then the user becomes frustrated.
2. Error Handling: Poor error message may result in rejecting the product.
3. Help Facilities: User requires help when he needs some information and he cannot find it then the user is in trouble.
4. Application Accessibility: It state whenever the application is simple to interact with or not. Special guidelines are given to user while interacting with software.

These slides are designed to accompany Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill, 2009). Slides copyright 2009 by Roger Pressman.